外部设备供应商开放平台数据传输字段级加密技术方案

目录

正文 1680 字（1.3 页） | 浏览 2 次（1 人） 全部 >

AI 速览

外部设备供应商开放平台数据传输字段级加密技术方案

1. 背景与目标

外部设备供应商通过开放平台接口向美团传输设备配置数据，其中部分字段（如 Wi-Fi 密码）属于敏感信息，不能明文传输，存在被网络抓包、日志泄露等风险。

本方案的目标是为敏感字段提供传输加密能力，使敏感数据不以明文形式出现在请求报文中。同时保持改动最小：不改变现有接口协议结构、不引入重放防护等额外机制，对接口 schema 零侵入。

2. 整体方案

采用字段级 AES-256-GCM 对称加密，每个供应商（按 appKey 维度）独立分配密钥，密钥通过线下安全渠道预共享，报文中不传输密钥。

整体流程如下：

1. 密钥分发：供应商在开放平台注册后，美团侧为该 appKey 生成 256 位 AES 密钥，通过线下安全渠道（或平台密钥管理模块）交付供应商；美团侧将密钥存入密钥管理服务（Squirrel/Cellar 或 KMS），不落代码仓库。
2. 加密（供应商侧）：仅对敏感字段（如 wifiPassword）加密。每次请求生成随机 12 字节 IV，使用 AES-256-GCM 加密字段明文，将 IV 与密文（含 16 字节认证标签）拼接后 Base64 编码，填回原字段。
3. 解密（美团侧）：服务端收到请求后，根据报文中的 appKey 从密钥管理服务取回对应密钥，从字段值中拆出 IV 与密文，执行 AES-GCM 解密还原明文，后续业务逻辑完全不变。

选择 AES-256-GCM 的理由：GCM 是带认证的加密模式，自带完整性校验（认证标签），密文被篡改或密钥不匹配时解密直接失败，无需额外引入 HMAC；相比 ECB/CBC 等模式不存在填充预言等已知问题；JDK 8+ 原生支持，供应商侧无第三方依赖。

3. 接口报文变化

加密前后报文对比（以设备 Wi-Fi 配置上报接口为例）：

加密前：

代码块 JSON

```json
{
  "isConfiguration": true,
  "roomNum": 10,
  "userConnectionType": "qrcode",
  "connectionType": "unified",
  "wifiAccount":"zhubawifi",
  "wifiPassword": "12345678"
}
```

加密后：

代码块 JSON

```json
{
  "isConfiguration": true,
  "roomNum": 10,
  "userConnectionType": "qrcode",
  "connectionType": "unified",
  "wifiAccount": "zhubawifi",
  "wifiPassword": "Wn3kQ9xT2aRfGh7uJmNp1sVbYc4EdA6L+zQ5tXoP",
  "encryptVersion": "v1"
}
```

说明：仅 wifiPassword 的值由明文变为 Base64(IV + 密文)，字段名与类型均不变。新增顶层可选字段 encryptVersion 用于标记加密版本，便于服务端区分密文与明文，并为后续密钥轮换预留扩展；该字段缺省时视为明文（兼容供应商灰度升级期间的存量流量）。

4. 密钥管理

密钥按 appKey 维度隔离，一个供应商一把密钥，避免跨供应商泄露面扩大。密钥生存周期分为生成、分发、存储、轮换四个环节。

生成环节由美团侧使用安全随机数生成器（SecureRandom / CSPRNG）生成 256 位密钥，禁止使用弱随机或人工指定口令。分发环节通过线下安全渠道交付，禁止经 IM、邮件等明文渠道传输。

存储环节：美团侧存入密钥管理服务并加密落盘，禁止硬编码在代码或配置文件中；供应商侧存入其自身的密钥管理系统，同样禁止硬编码。

密钥（Base64编码）： 0y/NPezSt+E6r1a0yXUCFVg3/HGcJTNymd5C5UTrpyA=

5. 加解密实现

加解密工具类（Java，美团侧与供应商侧共用同一套逻辑）：

代码块 Java

```java
public class FieldCrypto {

    private static final int IV_LENGTH = 12;
    private static final int TAG_LENGTH_BITS = 128;

    public static String encrypt(String plaintext, byte[] key) throws Exception {
        byte[] iv = new byte[IV_LENGTH];
        new SecureRandom().nextBytes(iv);

        Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
        cipher.init(Cipher.ENCRYPT_MODE, new SecretKeySpec(key, "AES"),
                new GCMParameterSpec(TAG_LENGTH_BITS, iv));
        byte[] cipherText = cipher.doFinal(plaintext.getBytes(StandardCharsets.UTF_8));

        // IV + 密文(含认证标签) 拼接后 Base64
        byte[] result = ByteBuffer.allocate(iv.length + cipherText.length)
```

```
                .put(iv).put(cipherText).array();
        return Base64.getEncoder().encodeToString(result);
    }

    public static String decrypt(String encoded, byte[] key) throws Exception {
        byte[] data = Base64.getDecoder().decode(encoded);
        byte[] iv = Arrays.copyOfRange(data, 0, IV_LENGTH);
        byte[] cipherText = Arrays.copyOfRange(data, IV_LENGTH, data.length);

        Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
        cipher.init(Cipher.DECRYPT_MODE, new SecretKeySpec(key, "AES"),
                new GCMParameterSpec(TAG_LENGTH_BITS, iv));
        return new String(cipher.doFinal(cipherText), StandardCharsets.UTF_8);
    }
}
```

美团服务端入口处解密示例：

代码块 Java

```
String key = keyManager.getKeyByAppKey(appKey); // 按 appKey 取密钥
String wifiPassword = FieldCrypto.decrypt(
        request.getWifiPassword(), key.getBytes(StandardCharsets.UTF_8));
```

关键实现约束：IV 必须使用密码学安全随机数生成器，且每次加密均重新生成，禁止固定 IV 或递增 IV 复用；GCM 认证标签长度固定为 128 bit，不要为节省体积降低标签长度。

6. 异常处理与可观测性

解密失败（密钥配置错误、密文被篡改、Base64 非法）会抛出 AEADBadTagException 或 IllegalArgumentException，服务端统一捕获并返回明确错误码 DECRYPT_FAILED，同时在日志中记录 appKey 与 encryptVersion（不记录密文原文），便于供应商排查密钥配置问题。

监控方面对解密失败率做告警，若单一供应商解密失败率突增，通常意味着密钥不一致或报文被篡改，需要介入排查。

7. 上线计划

第一阶段：美团侧上线解密逻辑（兼容明文与密文），并在密钥管理服务中完成密钥注册。此阶段不影响任何存量流量。

第二阶段：与供应商联调，供应商侧灰度开启加密，通过测试环境验证加解密全链路正确性。

第三阶段：供应商全量切换为加密传输，观察一周后美团侧可下线明文兼容逻辑（保留与否视存量数据回传情况决定）。

8. 风险与对策

密钥泄露风险：单供应商密钥泄露不影响其他供应商；通过 KMS 集中管理与轮换机制控制影响面。供应商实现错误风险：提供多语言参考实现与联调环境，上线前完成全链路验证。历史明文流量风险：切换完成前明文仍可见，需推动供应商尽快完成切换，并在切换完成后对日志中的历史明文做排查清理。

9. 非目标（Non-Goals）

本方案不解决以下问题，如有需要另行立项：整体报文签名与防篡改（沿用开放平台既有签名机制）；防重放攻击（明确不需要）；报文级全量加密（仅字段级）；密钥自动化协商分发（采用线下预共享，如后续需要可升级为 RSA 混合加密方案）。

仅供内部使用，未经授权，切勿外传